Data Science — szybki kurs Pythona

Część II. Wbudowane struktury danych i funkcje

Cel zajęć

Celem ćwiczeń jest poznanie najważniejszych struktur danych Pythona oraz konstrukcji, które będą często wykorzystywane podczas pracy z bibliotekami **NumPy** i **pandas**.

Po zajęciach powinieneś umieć:

- tworzyć i modyfikować listy,
- korzystać z indeksowania i wycinków,
- rozumieć różnicę między listą i krotką,
- korzystać ze słowników,
- iterować po danych,
- używać `enumerate()` i `zip()`,
- tworzyć listy za pomocą list comprehensions,
- definiować proste funkcje.

Każdy przykład wpisuj samodzielnie do kolejnej komórki Google Colab.

Komórkę uruchamiamy:

Shift + Enter

---

1. Lista — `list`

Lista pozwala przechowywać wiele wartości w jednym obiekcie.

Utwórz:

```
temperatures = [18.5, 21.0, 19.5, 24.0, 17.5]

print(temperatures)
print(type(temperatures))
```

Sprawdź liczbę elementów:

```
print(len(temperatures))
```

Małe zadanie

Utwórz listę:

```
scores = [72, 55, 91, 48, 83]
```

Wyświetl:

- całą listę,
- jej typ,
- liczbę elementów.

---

2. Indeksowanie listy

Elementy listy numerowane są od `0`.

```
languages = ["Python", "C++", "Java", "R"]

print(languages[0])
print(languages[1])
```

Sprawdź:

```
print(languages[-1])
print(languages[-2])
```

Ujemne indeksy pozwalają liczyć elementy od końca.

Małe zadanie

Dla:

```
values = [10, 20, 30, 40, 50]
```

wyświetl:

- pierwszy element,
- trzeci element,
- ostatni element.

---

3. Modyfikowanie listy

Lista jest obiektem modyfikowalnym.

```
numbers = [10, 20, 30]

numbers[1] = 200

print(numbers)
```

Dodaj element:

```
numbers.append(40)

print(numbers)
```

Usuń element:

```
numbers.remove(30)

print(numbers)
```

Małe zadanie

Utwórz:

```
cities = ["Warszawa", "Kraków", "Gdańsk"]
```

Następnie:

1. dodaj `"Poznań"`,
2. usuń `"Kraków"`,
3. wyświetl końcową listę.

---

4. Wycinki — slicing

Utwórz:

```
data = [10, 20, 30, 40, 50, 60]
```

Sprawdź:

```
print(data[1:4])
```

W zapisie:

```
data[1:4]
```

element o indeksie `1` należy do wyniku, natomiast element o indeksie `4` już nie.

Sprawdź:

```
print(data[:3])
print(data[3:])
```

Teraz:

```
print(data[::2])
```

oraz:

```
print(data[::-1])
```

Małe zadanie

Dla:

```
values = [1, 2, 3, 4, 5, 6, 7, 8]
```

wyświetl:

- pierwsze trzy elementy,
- co drugi element,
- całą listę w odwrotnej kolejności.

---

5. Lista a krotka

Lista:

```
numbers = [10, 20, 30]
```

jest modyfikowalna (inmuttable).

Krotka:

```
point = (10, 20)
```

jest niemodyfikowalna (muttable).

Sprawdź:

```
print(point)
print(type(point))
print(point[0])
```

Spróbuj:

```
point[0] = 100
```

Przeczytaj komunikat błędu.

---

6. Rozpakowywanie

Krotkę możemy rozpakować do kilku zmiennych:

```
point = (10, 20)

x, y = point

print(x)
print(y)
```

Sprawdź:

```
a = 5
b = 10

a, b = b, a

print(a)
print(b)
```

Małe zadanie

Dla:

```
student = ("Anna", 82)
```

rozpakuj dane do zmiennych:

```
name
score
```

i wyświetl je.

---

7. Słownik — `dict`

Słownik przechowuje dane w postaci:

klucz → wartość

Utwórz:

```
student = {
    "name": "Anna",
    "score": 82,
    "passed": True
}

print(student)
```

Odczytaj wartości:

```
print(student["name"])
print(student["score"])
```

Zmień wynik:

```
student["score"] = 90
```

Dodaj nową informację:

```
student["group"] = "Data Science"

print(student)
```

Małe zadanie

Utwórz:

```
measurement = {
    "city": "Warszawa",
    "temperature": 18.5,
    "humidity": 65
}
```

Następnie:

1. wyświetl temperaturę,
2. zmień ją na `20.0`,
3. dodaj klucz `"pressure"` z wartością `1013`.

---

8. Iterowanie po słowniku

Sprawdź:

```
for key in student:
    print(key)
```

Teraz:

```
for key, value in student.items():
    print(key, value)
```

Obserwacja

Metoda:

```
items()
```

pozwala jednocześnie uzyskać klucz i odpowiadającą mu wartość.

9. `enumerate()` i `zip()`

Mamy listę:

```
languages = ["Python", "C++", "R"]
```

Sprawdź:

```
for i, language in enumerate(languages):
    print(i, language)
```

`enumerate()` udostępnia jednocześnie indeks oraz wartość.

Mamy teraz dwie listy:

```
names = ["Anna", "Jan", "Ola"]
scores = [80, 65, 91]
```

Sprawdź:

```
for name, score in zip(names, scores):
    print(name, score)
```

`zip()` pozwala przechodzić równocześnie po odpowiadających sobie elementach kilku sekwencji.

Małe zadanie

Dane są:

```
cities = ["Warszawa", "Kraków", "Gdańsk"]
temperatures = [20.5, 22.0, 18.5]
```

Za pomocą `zip()` wypisz nazwę miasta i odpowiadającą mu temperaturę.

10. List comprehension

Załóżmy, że chcemy utworzyć listę kwadratów liczb.

Klasycznie:

```
squares = []

for x in range(5):
    squares.append(x ** 2)

print(squares)
```

Python pozwala zapisać to krócej:

```
squares = [x ** 2 for x in range(5)]

print(squares)
```

Taki zapis nazywamy **list comprehension**.

Teraz dodajmy warunek:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]

even = [x for x in numbers if x % 2 == 0]

print(even)
```

Małe zadanie

Dane są wyniki:

```
scores = [42, 75, 51, 33, 90, 68]
```

Utwórz listę zawierającą tylko wyniki wynoszące co najmniej `50`.

---

11. Funkcje — `def`

Utwórz:

```
def square(x):
    return x ** 2
```

Wywołaj:

```
print(square(5))
print(square(10))
```

Obserwacja

W definicji:

```
def square(x):
```

`x` jest argumentem funkcji.

Instrukcja:

```
return
```

zwraca wynik.

Funkcja może mieć kilka argumentów:

```
def average(a, b):
    return (a + b) / 2
```

Sprawdź:

```
print(average(10, 20))
```

Małe zadanie

Napisz funkcję:

```
def double(x):
```

która zwraca dwukrotność przekazanej liczby.

---

12. Funkcje przydatne przy pracy z danymi

Utwórz:

```
values = [10, 5, 17, 8, 20]
```

Sprawdź:

```
print(len(values))
print(min(values))
print(max(values))
print(sum(values))
```

Średnią możemy policzyć:

```
average = sum(values) / len(values)

print(average)
```

Małe zadanie

Dla:

```
temperatures = [18.5, 22.0, 19.0, 25.5, 17.5]
```

oblicz:

- liczbę pomiarów,
- temperaturę minimalną,
- temperaturę maksymalną,
- średnią temperaturę.

13. Ćwiczenie końcowe — mini analiza danych

Mamy dane:

```
students = [
    ("Anna", 78),
    ("Jan", 43),
    ("Ola", 91),
    ("Piotr", 68),
    ("Adam", 39)
]
```

Wyświetl wszystkie dane:

```
for name, score in students:
    print(name, score)
```

Zadanie 1

Wypisz tylko osoby, które uzyskały co najmniej `50` punktów.

Zadanie 2

Utwórz listę samych wyników:

```
scores = [score for name, score in students]
```

Wyświetl ją.

Zadanie 3

Oblicz:

- najmniejszy wynik,
- największy wynik,
- średni wynik.

Zadanie 4

Utwórz listę osób, które zaliczyły:

```
passed = [
    name
```

```
    for name, score in students
    if score >= 50
]
```

Wyświetl:

```
print(passed)
```

Policz liczbę osób, które zaliczyły:

```
print(len(passed))
```

---

14. Przeczytaj kod

Najpierw nie uruchamiaj programu.

```
data = [12, 5, 18, 7, 21, 4]

selected = [x for x in data if x >= 10]

minimum = min(selected)
maximum = max(selected)
average = sum(selected) / len(selected)

print(selected)
print(minimum)
print(maximum)
print(average)
```

Odpowiedz:

1. Jaką listę utworzy `selected`?
2. Jakie wartości zostaną pominięte?
3. Jaką wartość otrzyma `minimum`?
4. Jaką wartość otrzyma `maximum`?
5. Jaką wartość otrzyma `average`?

Dopiero teraz uruchom kod i sprawdź odpowiedzi.

Podsumowanie

Po wykonaniu ćwiczeń powinieneś rozumieć kod:

```
students = [
    ("Anna", 78),
    ("Jan", 43),
    ("Ola", 91),
    ("Piotr", 68)
]

passed = [
    name
    for name, score in students
    if score >= 50
]

print(passed)
```

Odpowiedz na pytania:

1. Czym różni się lista od krotki?
2. Jak odczytać pierwszy i ostatni element listy?
3. Co oznacza `data[1:4]`?
4. Do czego służy słownik?
5. Co robi `items()`?
6. Do czego służy `enumerate()`?
7. Do czego służy `zip()`?
8. Co oznacza list comprehension?
9. Do czego służy `def`?
10. Jak zwrócić wynik z funkcji?
11. Do czego służą `len()`, `min()`, `max()` i `sum()`?